

RAPPORT DU PROJET INTEGRE

Programmation Orientée Objet – Projet Intégré – Plateforme de Gestion
des Demandes PSH

3^e année Cybersécurité — École Supérieure d'Informatique et du
Numérique (ESIN)
Collège d'Ingénierie & d'Architecture (CIA)

Étudiants :

ACHOUKHANE Achraf
ATOURLABI Hibat Allah
CHEMAOU Aya
EL OUDRI Sara
HALLAL Yahya
HATHOUTI Mohammed Taha
JIDAL Ilyas
KABORE Mohammad Sharif Jonathan

Filière :

Cybersécurité

Année :

2025/2026

Enseignants :

Mme. Guermah, Mme. Hanifi & M. Rahhal

Date :

18 mai 2026

Remerciements

Ce projet a été réalisé dans le cadre du cours de Programmation Orientée Objet et du Projet Intégré dispensés par **Mme. Guermah**, **Mme. Hanifi** et **M. Rahhal** à l'UIR durant l'année universitaire 2025-2026.

Nous tenons à remercier nos encadrants pour la qualité de leur suivi et pour nous avoir proposé un projet ambitieux permettant d'appréhender la conception et le développement d'une application Java orientée objet complète, incluant une base de données relationnelle, une interface graphique Swing et des fonctionnalités d'accessibilité avancées.

Nous souhaitons également adresser nos sincères remerciements aux enseignants des formations antérieures :

- **M. Najib**, professeur de Programmation Orientée Objet au S5 à l'UIR, pour les fondements en POO Java qui ont directement nourri la conception de ce projet ;
- **M. Gruber**, professeur de POO aux S5 et S6 (2024-2025) à Polytech Grenoble, pour la maîtrise approfondie des concepts objet en Java ;
- **M. Gruber & M. Perin**, professeurs de Gestion de Projet Informatique (GPI) au S6 (2024-2025) à Polytech Grenoble, pour la méthodologie de conduite de projet qui a structuré notre démarche de développement.

Achraf ACHOUKHANE
Hibat Allah ATOURABI
Aya CHEMAOU
Sara EL OUDRI
Yahya HALLAL
Mohammed Taha HATHOUTI
Ilyas JIDAL
Mohammad Sharif Jonathan KABORE
Mai 2026

Table des matières

1	Introduction et Contexte du Projet	4
2	Architecture Globale du Projet	5
2.1	Organisation en couches	5
2.2	Plan d’adressage des packages	6
3	Modèle Objet — Diagramme de Classes	6
3.1	Diagramme de classes	6
3.2	Hiérarchie des classes	7
3.3	Classe abstraite <code>Utilisateur</code>	7
3.4	Classe <code>PSH</code> (extends <code>Utilisateur</code>)	8
3.5	Classe <code>Administrateur</code> (extends <code>Utilisateur</code>)	9
3.6	Classe abstraite <code>Dossier</code>	9
3.7	Classe <code>Demande</code> (extends <code>Dossier</code>)	10
3.8	Classe <code>Reclamation</code> (extends <code>Dossier</code>)	10
3.9	Énumérations	10
3.10	Classe <code>Archive</code>	10
3.11	Classe <code>Statistique</code>	11
4	Base de Données	11
4.1	Schéma relationnel	11
4.2	Points de conception notables	12
4.3	Singleton BD — Connexion JDBC	12
5	Interfaces Graphiques (Java Swing)	13
5.1	Thème d’accessibilité WCAG AAA	13
5.2	Module d’authentification — <code>LoginView</code> et <code>RegisterPage</code>	14
5.3	Espace étudiant PSH — <code>PshMainView</code>	15
5.4	Console administrateur — <code>AdminMainView</code>	16

6	Accessibilité — TTS et Reconnaissance Vocale	16
6.1	Synthèse vocale — TTSController	16
6.2	Reconnaissance vocale — VoskSpeechController	18
7	Diagrammes Complémentaires	19
7.1	Diagramme de cas d'utilisation	19
7.2	Diagramme de séquence — Soumission d'une demande	20
7.3	Diagramme de séquence — Lister des demandes à partir des Archives	20
7.4	Diagramme d'états — Cycle de vie d'un Dossier	21
8	Difficultés Rencontrées et Solutions	21
8.1	Chemins de fichiers incorrects (Vosk et TTS)	21
8.2	Hallucinations du modèle Vosk	22
8.3	Superposition de flux audio	22
8.4	Conflits Git entre Eclipse et IntelliJ IDEA	23
	Conclusion	24
	Références	25

1 Introduction et Contexte du Projet

L’Université Internationale de Rabat (UIR) accueille des étudiants en situation de handicap (PSH) qui nécessitent des aménagements académiques spécifiques : tiers-temps aux examens, accessibilité des locaux, accompagnement humain ou mise à disposition de matériel adapté. La gestion de ces demandes, historiquement manuelle et fragmentée, constitue un frein à la réactivité et à la traçabilité des décisions de la commission.

Ce projet intégré vise à concevoir et développer une **plateforme logicielle complète** permettant :

- aux **étudiants PSH** de soumettre, suivre et contester des demandes d’aménagement ;
- aux **administrateurs** (service scolarité) d’instruire, décider et archiver ces dossiers ;
- de garantir une **accessibilité maximale** grâce à des interfaces conformes WCAG AAA, une synthèse vocale (TTS) et une reconnaissance vocale hors-ligne (Vosk).

Ce projet a été réalisé en appliquant les principes fondamentaux de la Programmation Orientée Objet : héritage, polymorphisme, interfaces, encapsulation et séparation des responsabilités, avec :

- **Langage / Framework** : Java 11, Swing ;
- **Base de données** : MySQL 8, JDBC ;
- **Bibliothèques tierces** : Vosk (reconnaissance vocale hors-ligne), mpg123 / PowerShell / afplay (TTS selon l’OS), `mysql-connector-j-8.0.33`, `json-simple-1.1.1`, `jna-5.13.0`, `jbCrypt-0.4` ;
- **Outils** : Eclipse IDE, IntelliJ IDEA, Git.

2 Architecture Globale du Projet

2.1 Organisation en couches

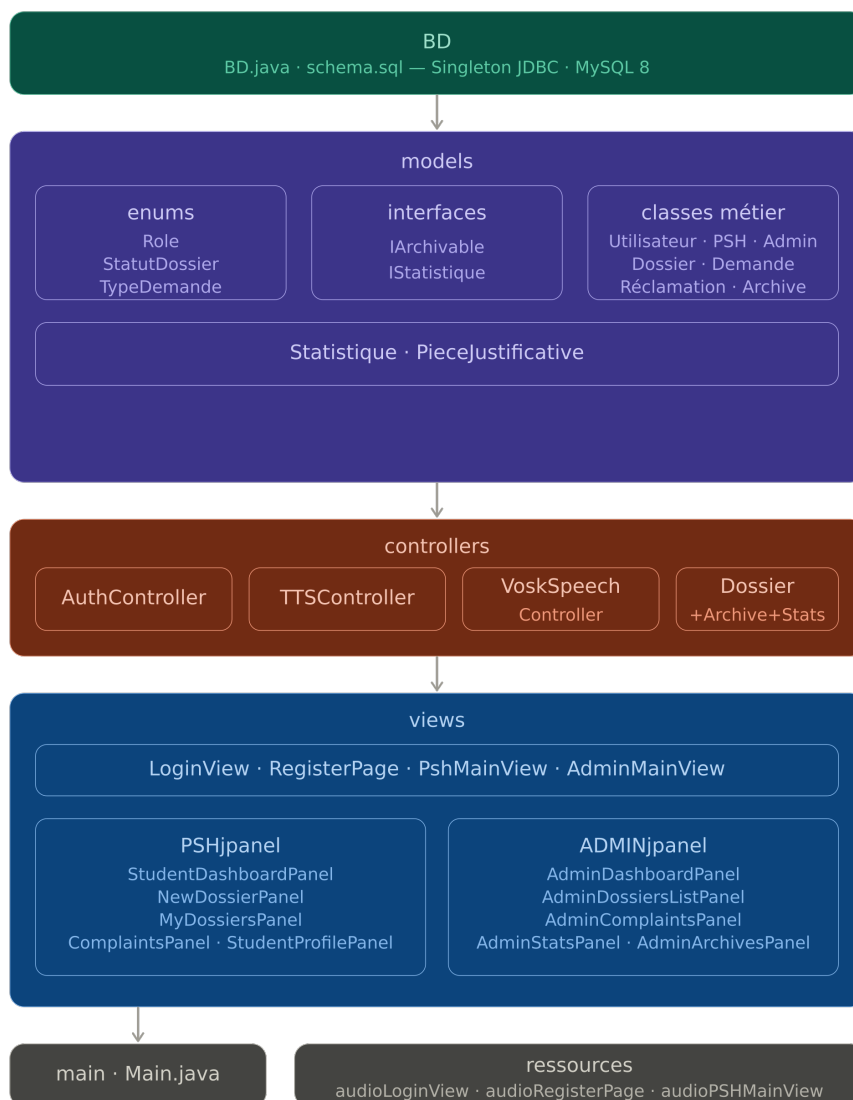


FIGURE 1 – Architecture en couches de la plateforme PSH

L'application suit une architecture en quatre couches distinctes :

1. **Couche Modèle (models)** : classes métier représentant les entités du domaine (Utilisateur, Dossier, Demande, Reclamation, Archive, Statistique, PieceJustificative);
2. **Couche Contrôleurs (controllers)** : logique transversale non liée à l'affichage (AuthController, TTSController, VoskSpeechController, DossierController, DemandeController, ReclamationController, StatistiqueController, ArchiveController);
3. **Couche Vues (views)** : interfaces graphiques Swing (LoginView, RegisterPage, PshMainView, AdminMainView et tous les panels PSH et Admin);

4. **Couche Persistance (BD) :** accès JDBC à la base de données MySQL via le singleton BD.

2.2 Plan d'adressage des packages

Package	Sous-pkg	Contenu
models	—	Classes métier et énumérations
models	enums	Role, StatutDossier, TypeDemande
models	interfaces	IArchivable, IStatistique
controllers	—	AuthController, TTSController, VoskSpeechController, DossierController, DemandeController, ReclamationController, StatistiqueController, ArchiveController
views	—	LoginView, RegisterPage, PshMainView, AdminMainView
views	PSHjpanel	Panels espace étudiant
views	ADMINjpanel	Panels console administrateur
BD	—	Singleton BD
main	—	Point d'entrée Main

3 Modèle Objet — Diagramme de Classes

3.1 Diagramme de classes

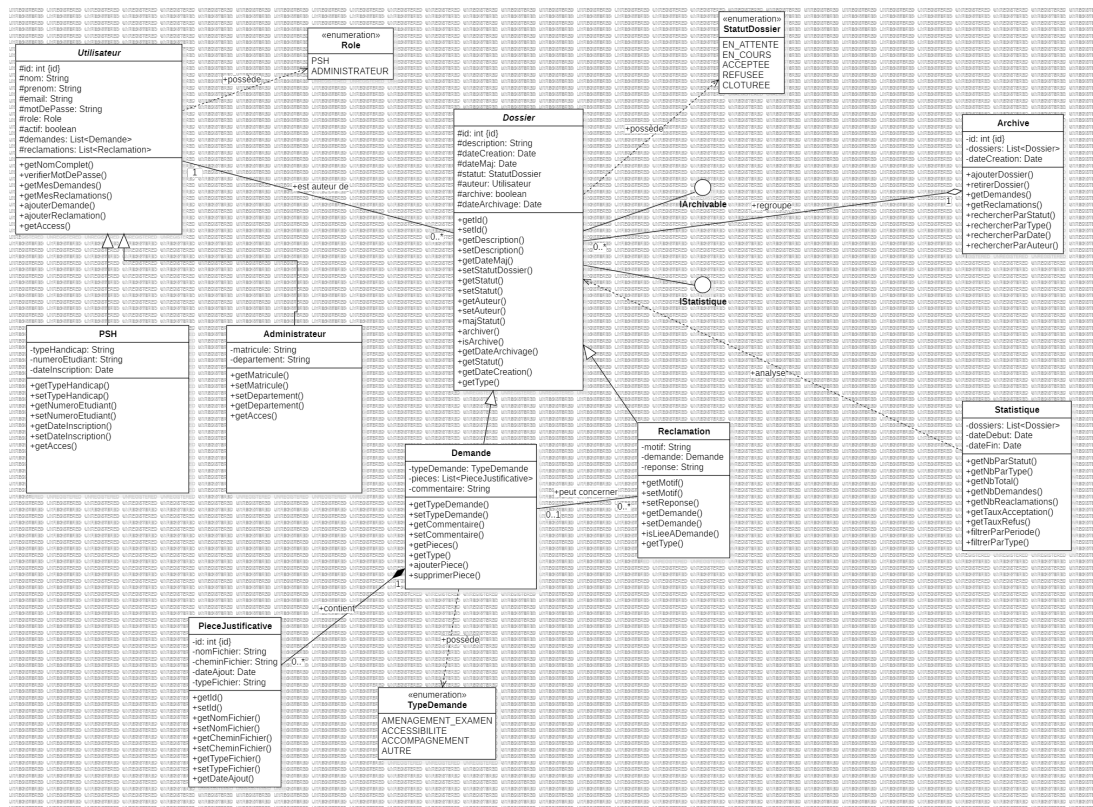


FIGURE 2 – Diagramme de classes UML de la plateforme PSH

3.2 Hiérarchie des classes

La hiérarchie repose sur deux arbres d'héritage principaux :

- **Utilisateur** $\rightarrow$ {**PSH**, **Administrateur**} : les deux acteurs du système, partageant les attributs communs mais différenciés par leurs droits d'accès et leurs attributs métier ;
- **Dossier** $\rightarrow$ {**Demande**, **Reclamation**} : les deux types de dossiers traités par la plateforme, partageant le cycle de vie (statut, archivage, horodatages) mais différenciés par leur contenu.

3.3 Classe abstraite Utilisateur

Utilisateur est la racine de la hiérarchie des acteurs. Elle encapsule les attributs communs, définit la méthode abstraite `getAcces()`, et expose l'ensemble des méthodes métier utilisées par les contrôleurs et les vues.

Attribut / Méthode	Type	Description
id	int	Identifiant unique (clé primaire BD)
nom, prenom	String	Identité civile
email	String	Identifiant de connexion (UNIQUE)
motDePasse	String	Mot de passe de connexion
role	Role	PSH ou ADMINISTRATEUR
actif	boolean	Compte activé ou non
demandes	List<Demande>	Demandes soumises
reclamations	List<Reclamation>	Réclamations soumises
getNomComplet()	String	Retourne <code>prenom + " " + nom</code>
verifierMotDePasse()	boolean	Vérifie le mot de passe saisi
getMesDemandes()	List<Demande>	Retourne la liste des demandes
getMesReclamations()	List<Reclamation>	Retourne la liste des réclamations
ajouterDemande()	void	Ajoute une demande à la liste
ajouterReclamation()	void	Ajoute une réclamation à la liste
getAcces()	String[]	<i>Abstraite</i> — liste des droits

Listing 1 – Classe abstraite Utilisateur — attributs, méthodes métier et méthode abstraite

```
1 public abstract class Utilisateur {
2     protected int id;
3     protected String nom, prenom, email, motDePasse;
4     protected Role role;
5     protected boolean actif;
6     protected List<Demande> demandes;
7     protected List<Reclamation> reclamations;
8
9     public Utilisateur(String nom, String prenom, String email,
10                        String motDePasse, Role role) {
11         this.nom = nom; this.prenom = prenom;
12         this.email = email; this.motDePasse = motDePasse;
13         this.role = role; this.actif = true;
14         this.demandes = new ArrayList<Demande>();
```



```

15         this.reclamations = new ArrayList<Reclamation>();
16     }
17
18     /* --- Methodes metier --- */
19     public String getNomComple() {
20         return this.prenom + " " + this.nom;
21     }
22     public boolean verifierMotDePasse(String mdp) {
23         return this.motDePasse.equals(mdp);
24     }
25     public List<Demande> getMesDemandes() { return this.
        demandes; }
26     public List<Reclamation> getMesReclamations() { return this.
        reclamations; }
27     public void ajouterDemande(Demande d) { this.demandes.
        add(d); }
28     public void ajouterReclamation(Reclamation r) { this.
        reclamations.add(r); }
29
30     public abstract String[] getAcces();
31 }

```

3.4 Classe PSH (extends Utilisateur)

La classe PSH représente un étudiant en situation de handicap. Elle ajoute : `typeHandicap`, `numeroEtudiant` et `dateInscription`. Sa méthode `getAcces()` retourne les droits propres à un étudiant.

Listing 2 – Classe PSH — attributs spécifiques et droits d'accès

```

1 public class PSH extends Utilisateur {
2     private String typeHandicap;
3     private String numeroEtudiant;
4     private Date dateInscription;
5
6     public PSH(String nom, String prenom, String email,
7         String motDePasse, String typeHandicap,
8         String numeroEtudiant) {
9         super(nom, prenom, email, motDePasse, Role.PSH);
10        this.typeHandicap = typeHandicap;
11        this.numeroEtudiant = numeroEtudiant;
12        this.dateInscription = new Date();
13    }
14
15    @Override
16    public String[] getAcces() {
17        return new String[]{
18            "SOUMETTRE_DEMANDE",          "CONSULTER_DEMANDE",
19            "MODIFIER_DEMANDE",           "SUPPRIMER_DEMANDE",
20            "SOUMETTRE_RECLAMATION",      "SUIVRE_RECLAMATION",

```

```

21         "LISTER_HISTORIQUE_DEMANDES", "
22         LISTER_HISTORIQUE_RECLAMATIONS "
23     };
24 }

```

3.5 Classe Administrateur (extends Utilisateur)

La classe `Administrateur` représente un agent du service scolarité. Elle ajoute `matricule` et `departement`. Son périmètre d'accès couvre la validation de comptes, l'instruction des demandes, le traitement des réclamations, la consultation des statistiques et la gestion des archives.

3.6 Classe abstraite Dossier

`Dossier` est la classe mère commune aux `Demandes` et aux `Reclamations`. Elle implémente deux interfaces :

- `IArchivable` : contrat d'archivage — `archiver()`, `isArchive()`, `getDateArchivage()` ;
- `IStatistique` : contrat de métriques — `getStatut()`, `getType()`, `getDateCreation()`.

Attribut	Type	Description
id	int	Identifiant unique
description	String	Description textuelle du dossier
dateCreation	Date	Date de création
dateMaj	Date	Date de dernière mise à jour
statut	StatutDossier	EN_ATTENTE, EN_COURS, ACCEPTEE, REFUSEE, CLOTUREE
auteur	Utilisateur	Référence vers le créateur
archive	boolean	Dossier archivé ou non
dateArchivage	Date	Date d'archivage (null si actif)

Listing 3 – Dossier — implémentation `IArchivable` et `majStatut()`

```

1  @Override
2  public void archiver() {
3      this.archive      = true;
4      this.dateArchivage = new Date();
5      this.statut       = StatutDossier.CLOTUREE;
6  }
7
8  public void majStatut(StatutDossier statut) {
9      this.statut = statut;
10     this.dateMaj = new Date();
11 }
12
13 public abstract String getType();

```

3.7 Classe Demande (extends Dossier)

Une *Demande* est une demande d'aménagement soumise par un étudiant PSH. Elle enrichit *Dossier* avec :

- `typeDemande` : enum `TypeDemande` (`AMENAGEMENT_EXAMEN`, `ACCESSIBILITE`, `ACCOMPAGNEMENT`, `AUTRE`);
- `pieces` : liste de `PieceJustificative` (certificats médicaux, attestations MDPH...);
- `commentaire` : annotation laissée par l'administrateur après instruction.

La méthode `getType()` retourne `typeDemande.name()`, permettant à la classe *Statistique* de filtrer les dossiers par type sans *instanceof*.

3.8 Classe Reclamation (extends Dossier)

Une *Reclamation* est la contestation d'une décision. Elle enrichit *Dossier* avec `motif`, `reponse` et une référence optionnelle vers la *Demande* contestée.

Listing 4 – Reclamation — deux constructeurs et `isLieeADemande()`

```
1 // Reclamation liee a une demande specifique
2 public Reclamation(String description, Utilisateur auteur,
3                     Demande demande, String motif) {
4     super(description, auteur);
5     this.demande = demande;
6     this.motif    = motif;
7     this.reponse  = "";
8 }
9
10 // Reclamation generale (sans demande cible)
11 public Reclamation(String description, Utilisateur auteur,
12                     String motif) {
13     super(description, auteur);
14     this.demande = null;
15     this.motif   = motif;
16     this.reponse = "";
17 }
18
19 public boolean isLieeADemande() { return this.demande != null; }
```

3.9 Énumérations

Enum	Valeurs
Role	PSH, ADMINISTRATEUR
StatutDossier	EN_ATTENTE, EN_COURS, ACCEPTEE, REFUSEE, CLOTUREE
TypeDemande	AMENAGEMENT_EXAMEN, ACCESSIBILITE, ACCOMPAGNEMENT, AUTRE

3.10 Classe Archive

La classe *Archive* maintient une collection de *Dossiers* archivés. L'ajout via `ajouterDossier()` déclenche automatiquement `archiver()` sur le dossier, fixant le statut à `CLOTUREE`. Elle

expose des méthodes de recherche multicritères — par statut, type, date et auteur — déléguées en BD par `ArchiveController`.

3.11 Classe Statistique

La classe `Statistique` opère sur une liste de `Dossiers` pour une période donnée. Elle fournit :

- `getNbTotal()`, `getNbDemandes()`, `getNbReclamations()` : comptages globaux;
- `getNbParStatut(StatutDossier)`, `getNbParType(String)` : comptages filtrés;
- `getTauxAcceptation()`, `getTauxRefus()` : indicateurs de performance en pourcentage;
- `filtrerParPeriode(Date, Date)`, `filtrerParType(String)` : sous-ensembles filtrés.

Séparation des responsabilités

`StatistiqueController` ne calcule rien : il charge les dossiers depuis la BD, construit un objet `Statistique`, et laisse le modèle effectuer tous les calculs. `AdminStatsPanel` appelle `StatistiqueController.getStatistiques(debut, fin)` pour un filtre périodique, et `getStatistiquesGlobales()` pour le tableau de bord.

Listing 5 – Statistique — taux d’acceptation et filtrage par période

```
1 public double getTauxAcceptation() {
2     int nbDm = getNbDemandes();
3     if (nbDm == 0) return 0.0;
4     return (double) getNbParStatut(StatutDossier.ACCEPTEE) / nbDm *
        100;
5 }
6
7 public List<Dossier> filtrerParPeriode(Date debut, Date fin) {
8     List<Dossier> resultat = new ArrayList<Dossier>();
9     for (Dossier d : this.dossiers) {
10         Date date = d.getDateCreation();
11         if (!date.before(debut) && !date.after(fin))
12             resultat.add(d);
13     }
14     return resultat;
15 }
```

4 Base de Données

4.1 Schéma relationnel

La base de données `projetintegre` adopte un schéma *table-per-subclass* pour mapper fidèlement la hiérarchie objet :

Table	Clé primaire	Description
utilisateur	idUtilisateur AUTO_INC.	Table mère — attributs communs
psh	idUtilisateur FK	typeHandicap, numeroEtudiant, dateInscription
administrateur	idUtilisateur FK	matricule, departement
dossier	idDossier AUTO_INC.	Table mère — attributs communs
demande	idDossier FK	typeDemande, commentaire
reclamation	idDossier FK	motif, reponse, idDemande FK
pieceJustificative	idPiece AUTO_INC.	nomFichier, cheminFichier, typeFichier

4.2 Points de conception notables

- **ON DELETE CASCADE** sur toutes les FK enfants : la suppression d'un utilisateur supprime automatiquement ses dossiers et ses pièces justificatives ;
- **ON DELETE RESTRICT** sur `fk_reclamation_demande` : une demande ne peut être supprimée si une réclamation y est liée, protégeant la cohérence référentielle ;
- Index sur `dossier.statut` et `dossier.dateCreation` pour optimiser les requêtes de filtrage ;
- La colonne `statut` est de type `VARCHAR(20)` et non `ENUM MySQL`, facilitant l'ajout de nouveaux statuts sans migration de schéma.

4.3 Singleton BD — Connexion JDBC

Listing 6 – Singleton BD — gestion de la connexion JDBC

```

1 public class BD {
2     private static final String URL =
3         "jdbc:mysql://localhost:3306/projetintegre";
4     private static final String USER = "root";
5     private static final String PASSWORD = "";
6     private static Connection conn = null;
7
8     public static Connection getConnection() {
9         try {
10             if (conn == null || conn.isClosed()) {
11                 Class.forName("com.mysql.cj.jdbc.Driver");
12                 conn = DriverManager.getConnection(URL, USER,
13                     PASSWORD);
14                 System.out.println("Connexion BD établie.");
15             }
16         } catch (ClassNotFoundException e) {
17             System.err.println("Driver introuvable : " + e.
18                 getMessage());
19         } catch (SQLException e) {

```

```

18         System.err.println("Erreur BD: " + e.getMessage());
19     }
20     return conn;
21 }
22 }

```

Pattern Singleton

Le pattern Singleton garantit qu'une seule instance de connexion JDBC est créée et partagée entre toutes les vues. La double vérification `conn == null || conn.isClosed()` assure la robustesse lors de reconnections successives. Tous les contrôleurs appellent `BD.getConnection()` sans instancier leur propre connexion.

5 Interfaces Graphiques (Java Swing)

5.1 Thème d'accessibilité WCAG AAA

L'application propose deux thèmes basculables à tout moment via un bouton dédié dans chaque vue :

Élément	Valeur	Justification
Fond standard	#F9FAFB (gris clair)	Relief visuel sans éblouissement
Fond accessibilité	#111827 (noir profond)	Absorption maximale de la lumière
Texte accessibilité	#FDE047 (jaune vif)	Ratio de contraste > 7 : 1
Contours	Bordures 2 px jaune	Délimitation des zones interactives
Boutons	<code>setContentAreaFilled(false)</code>	Neutralisation des dégradés OS natifs
Bascule	Bouton « Mode Contraste Élevé »	Choix utilisateur préservé en session

La méthode `applyThemeRecursively()` parcourt récursivement l'arborescence Swing pour appliquer le thème uniformément, y compris sur les composants instanciés dynamiquement. Les captures ci-dessous illustrent la bascule de thème sur la `LoginView` et la console `AdminMainView` :

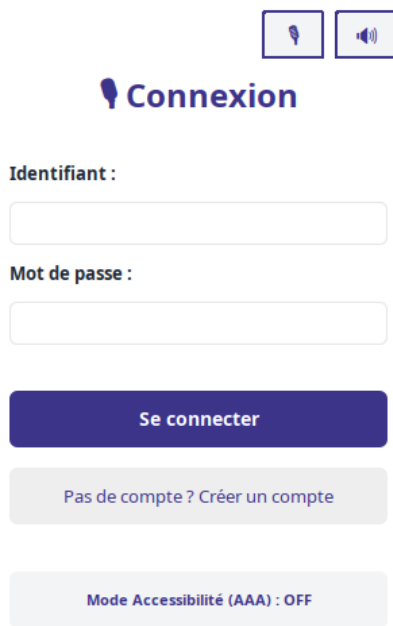


FIGURE 3 – LoginView — thème standard

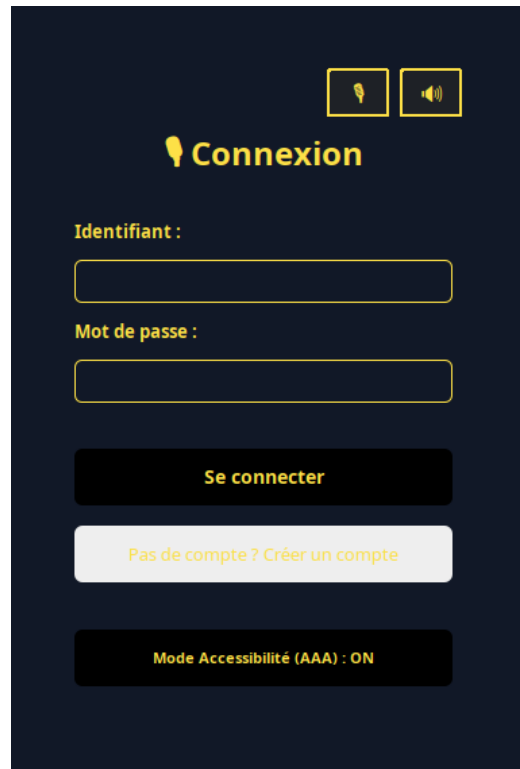


FIGURE 4 – LoginView — thème accessibilité

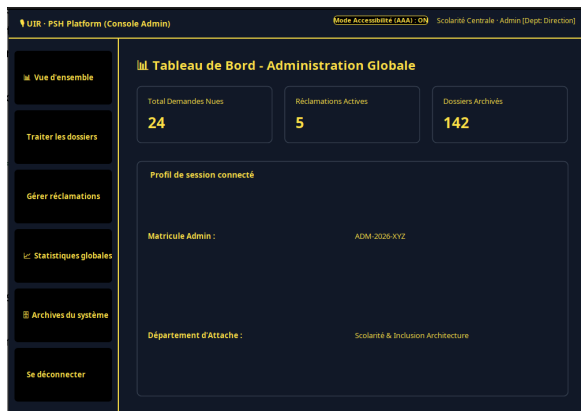


FIGURE 5 – AdminMainView — thème standard

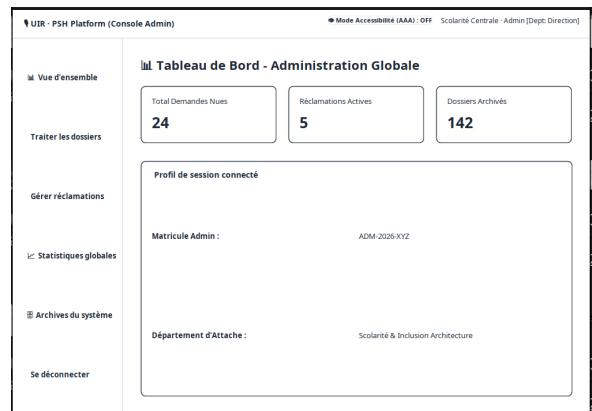


FIGURE 6 – AdminMainView — thème accessibilité

5.2 Module d'authentification — LoginView et RegisterPage

Le module d'authentification regroupe deux écrans permutés par un `CardLayout` sans recréer la fenêtre.

LoginView

L'écran de connexion présente :

- Un champ email et un champ mot de passe ;

- Un bouton de connexion déléguant l'authentification à `AuthController.connexion(email, mdp)`, qui exécute une **requête préparée** (`PreparedStatement`) protégeant contre les injections SQL;
- Un routage par rôle via un `switch` sur `u.getRole() : PSH → PshMainView, ADMINISTRATEUR → AdminMainView`;
- Un bouton microphone déclenchant `VoskSpeechController.startListening()`;
- Un bouton mute basculant `TTSToggleController.toggleTTS()`;
- Des événements `mouseenter` sur chaque champ et bouton jouant le fichier audio correspondant (sous-dossier `audioLoginView` par défaut).

Listing 7 – LoginView — routage par rôle via AuthController

```

1 Utilisateur u = AuthController.connexion(email, mdp);
2
3 if (u == null) {
4     JOptionPane.showMessageDialog(LoginView.this,
5         "Email ou mot de passe incorrect.",
6         "Echec de connexion", JOptionPane.ERROR_MESSAGE);
7     return;
8 }
9
10 dispose();
11 switch (u.getRole()) {
12     case PSH:                new PshMainView();    break;
13     case ADMINISTRATEUR:    new AdminMainView();   break;
14 }

```

RegisterPage

La page d'inscription collecte : nom, prénom, email, numéro étudiant, type de handicap, mot de passe (avec confirmation) et un justificatif médical (PDF via `JFileChooser`). La validation métier vérifie le format de l'email, la concordance des deux mots de passe et la présence du PDF avant d'appeler `AuthController.inscription()`. Le compte créé est inactif (`actif = FALSE`) jusqu'à validation par un administrateur. Chaque champ déclenche un guide audio via `TTSToggleController.playSound("audioRegisterPage", ...)`.

5.3 Espace étudiant PSH — PshMainView

La vue principale PSH utilise un `CardLayout`. La *sidebar* est construite dynamiquement à partir de trois tableaux parallèles (labels, actions, fichiers audio), ce qui rend triviale l'extension future de la navigation :

Listing 8 – PshMainView — construction dynamique de la sidebar avec audio

```

1 String[] labels = {"Tableau de bord", "Nouvelle Demande",
2                   "Mes Dossiers", "Reclamations",
3                   "Mon Profil", "Se deconnecter"};
4 String[] actions = {"DASHBOARD", "NEW_DOSSIER", "MY_DOSSIERS",
5                     "COMPLAINTS", "PROFILE", "LOGOUT"};
6 String[] audios = {"zone_dashboard.mp3", "
zone_nouvelledemande.mp3",

```



```

7         "zone_dossiers.mp3",          "zone_reclamations
8         .mp3",
9         "zone_profil.mp3",          "btn_sedeconnecter
10        .mp3"};
11 // AUDIO_DIR = "audioPSHMainView"
12 btn.addMouseListener(new MouseAdapter() {
13     @Override public void mouseEntered(MouseEvent e) {
14         TTSTController.playSound(AUDIO_DIR, audioFile);
15     }
16 });

```

Panel	Classe	Fonctionnalité
Tableau de bord	StudentDashboardPanel	Statistiques personnelles, dossiers récents
Nouvelle demande	NewDossierPanel	Formulaire : type, description, upload de pièce
Mes dossiers	MyDossiersPanel	Historique complet des demandes et réclamations
Réclamations	ComplaintsPanel	Soumission d'une réclamation liée à un dossier
Mon profil	StudentProfilePanel	Données personnelles (attributs PSH)

5.4 Console administrateur — AdminMainView

Panel	Classe	Fonctionnalité
Vue d'ensemble	AdminDashboardPanel	Compteurs globaux via StatistiqueController
Traiter dossiers	AdminDossiersListPanel	Accepter, Refuser, Commenter un dossier
Gérer réclamations	AdminComplaintsPanel	Réclamations reçues, saisie de réponse
Statistiques	AdminStatsPanel	Métriques et filtrage périodique
Archives	AdminArchivesPanel	Recherche multicritères via ArchiveController

6 Accessibilité — TTS et Reconnaissance Vocale

6.1 Synthèse vocale — TTSTController

Le `TTSTController` gère la lecture audio des éléments de l'interface via une API entièrement statique. La version finale adopte une **architecture à sous-dossiers** : chaque vue déclare sa propre constante `AUDIO_DIR` et passe le sous-dossier à la surcharge générale.

Sous-dossier	Fichiers audio
audioLoginView	zone_identifiant.mp3, zone_mdp.mp3, btn_seconnecter.mp3, btn_creercompte.mp3
audioRegisterPage	zone_nom.mp3, zone_prenom.mp3, zone_email.mp3, zone_id.mp3, zone_handicap.mp3, zone_mdp1.mp3, zone_mdp2.mp3, btn_justificatif.mp3, btn_sinscrire.mp3, btn_dejacompte.mp3
audioPSHMainView	zone_dashboard.mp3, zone_nouvelledemande.mp3, zone_dossiers.mp3, zone_reclamations.mp3, zone_profil.mp3, btn_sedeconnecter.mp3, btn_contraste.mp3

La surcharge `playSound(fileName)` redirige vers `audioLoginView` pour la compatibilité descendante de `LoginView`. Sur Windows, les `.mp3` sont automatiquement substitués par leur équivalent `.wav`.

Listing 9 – TTSTController — double surcharge et lecture multiplateforme

```

1 // Surcharge de commodite (LoginView)
2 public static void playSound(String fileName) {
3     playSound("audioLoginView", fileName);
4 }
5
6 // Surcharge generale
7 public static void playSound(String subfolder, String fileName) {
8     if (!ttsEnabled) return;
9     new Thread(() -> {
10         try {
11             if (currentAudio != null && currentAudio.isAlive())
12                 currentAudio.destroy();
13
14             String base = System.getProperty("user.dir");
15             String os = System.getProperty("os.name").
16                 toLowerCase();
17             String resBase = base + File.separator + "src"
18                 + File.separator + "ressources";
19             String path;
20             if (os.contains("win"))
21                 path = resBase + File.separator + subfolder
22                     + File.separator + fileName.replace(".mp3", ".
23                     wav");
24             else
25                 path = resBase + File.separator + subfolder
26                     + File.separator + fileName;
27
28             File f = new File(path);
29             if (!f.exists()) {
30                 System.err.println("[AUDIO] Fichier introuvable : "
31                     + path);
32                 return;
33             }
34         }
35     })
36 }

```

```

31         if (os.contains("linux"))
32             currentAudio = new ProcessBuilder("mpg123", "-q",
33                 path)
34                                     .inheritIO().start();
35         else if (os.contains("win"))
36             currentAudio = new ProcessBuilder("powershell", "-c",
37                 "(New-Object Media.SoundPlayer'" + path +
38                 "') .PlaySync()").start();
39         else
40             currentAudio = new ProcessBuilder("afplay", path).
41                 start();
42             currentAudio.waitFor();
43     } catch (Exception e) {
44         System.err.println("Erreur_système_audio:_ " + e.
45             getMessage());
46     }
47 }

```

6.2 Reconnaissance vocale — VoskSpeechController

Le VoskSpeechController utilise la bibliothèque **Vosk** (modèle `vosk-model-small-en-us-0.15`) pour la reconnaissance vocale entièrement hors-ligne. Il implémente un double filtre de sécurité :

1. **Grammaire JSON Vosk** : restreint la reconnaissance à un vocabulaire prédéfini ;
2. **Liste blanche Java** (`ALLOWED_COMMANDS`) : filtre supplémentaire pour rejeter tout résultat hors dictionnaire.

Listing 10 – VoskSpeechController — double filtre de sécurité

```

1 private static final List<String> ALLOWED_COMMANDS = Arrays.asList(
2     "admin", "root", "login", "next", "password",
3     "exit", "connect", "username", "send"
4 );
5
6 if (recognizer.acceptWaveForm(buffer, bytesRead)) {
7     String text = parseVoskJson(recognizer.getResult())
8         .trim().toLowerCase();
9     if (!text.isEmpty() && ALLOWED_COMMANDS.contains(text)) {
10         System.out.println("[VOSK]_Commande_validee:_ " + text);
11         listener.onResult(text);
12     } else if (!text.equals("[unk]") && !text.isEmpty()) {
13         System.out.println("[VOSK]_Mot_hors_dictionnaire_rejete:_ "
14             + text);
15     }
16 }

```

Initialisation asynchrone du modèle Vosk

Le modèle est chargé dans un thread séparé dès le démarrage (`Main.java`) pour ne pas bloquer l’affichage de l’écran de connexion. `initModel()` sonde deux emplacements : `workspace/lib/` puis `lib/` (relatif à `user.dir`).

7 Diagrammes Complémentaires

7.1 Diagramme de cas d’utilisation

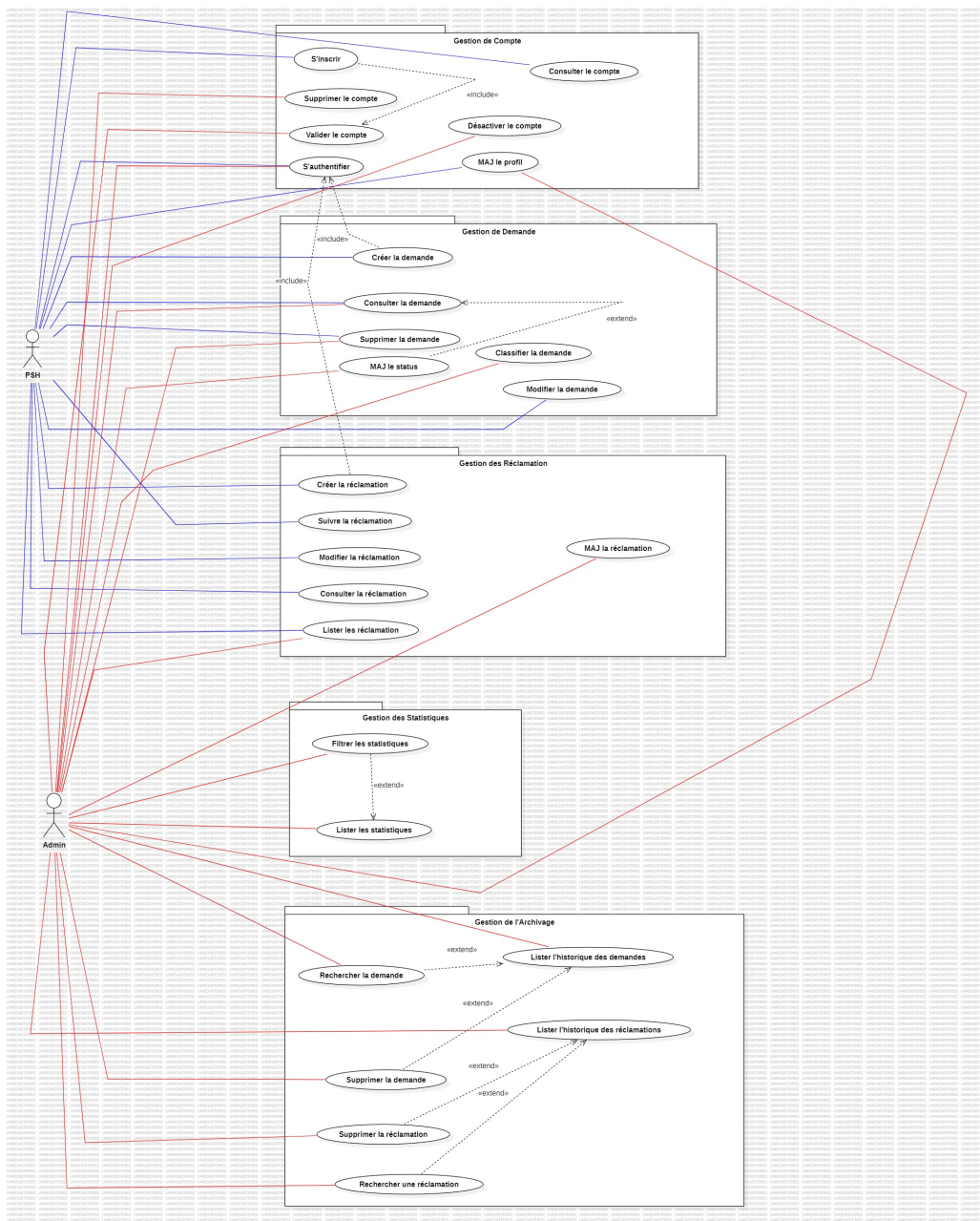


FIGURE 7 – Diagramme de cas d’utilisation — acteurs PSH et Administrateur

7.2 Diagramme de séquence — Soumission d'une demande

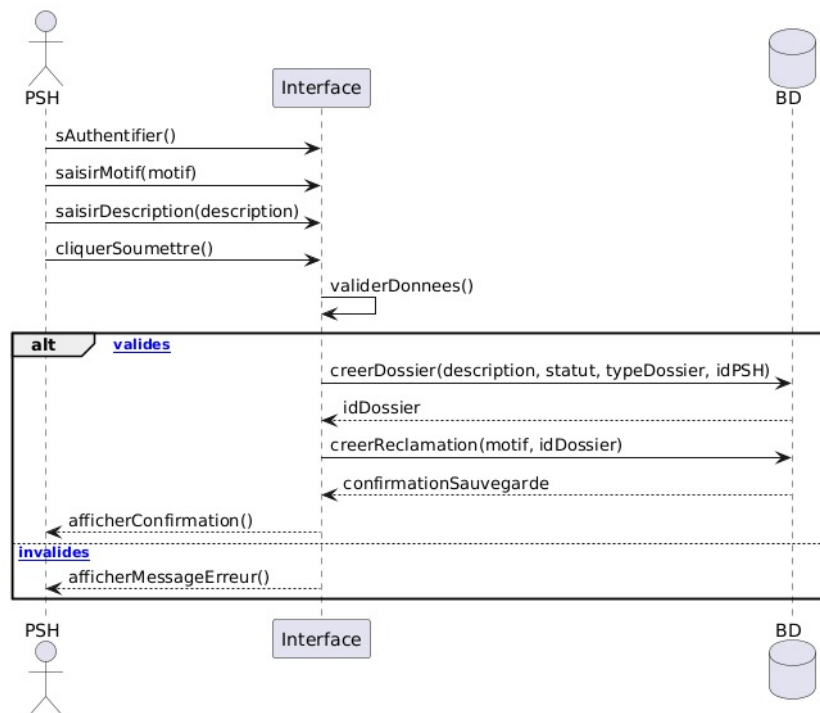


FIGURE 8 – Séquence de soumission d'une demande d'aménagement

7.3 Diagramme de séquence — Lister des demandes à partir des Archives

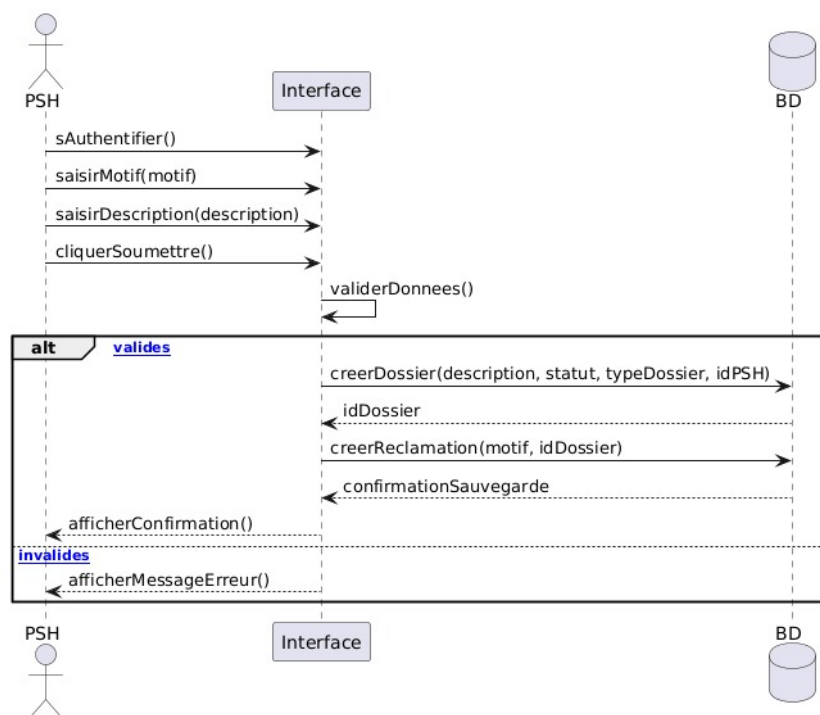


FIGURE 9 – Séquence de listage des demandes à partir des Archives

7.4 Diagramme d'états — Cycle de vie d'un Dossier

Le cycle de vie d'un dossier suit la progression des statuts de l'énumération `StatutDossier`. L'archivage fixe automatiquement le statut à `CLOTUREE` via la méthode `archiver()` de l'interface `IArchivable` :

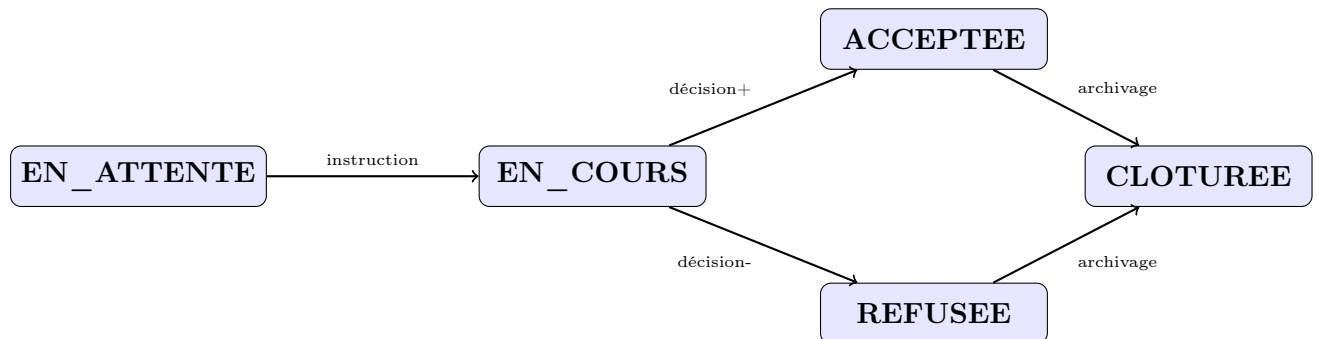


FIGURE 10 – Cycle de vie d'un Dossier — transitions `StatutDossier`

8 Difficultés Rencontrées et Solutions

8.1 Chemins de fichiers incorrects (Vosk et TTS)

Bug — Modèle Vosk et fichiers audio introuvables au démarrage

Symptôme : au lancement, la console affichait :

```
[VOSK] ERREUR : Modèle introuvable à :  
.../workspace/projetintegre/src/ressources/vosk-model-small-en-us-0.15  
[AUDIO] Fichier introuvable :  
.../src/projetintegre/ressources/audioLoginView/...
```

Cause : `System.getProperty("user.dir")` retourne `workspace/projetintegre/` depuis Eclipse. Les chemins codés ajoutaient `workspace/` en préfixe, créant une duplication.

Fix — Correction dans `TTSTController` et `VoskSpeechController`

TTSTController : suppression du préfixe redondant :

```
// AVANT (incorrect)  
String resBase = base + File.separator + "workspace"  
                + File.separator + "projetintegre"  
                + File.separator + "src" + File.separator + "  
                ressources";  
  
// APRES (correct)  
String resBase = base + File.separator + "src"  
                + File.separator + "ressources";
```

VoskSpeechController : double sonde pour les deux configurations :

```
File modelDir = new File(base,
```

```

        "workspace" + File.separator + "lib" + File.separator +
        folderName);
if (!modelDir.exists())
    modelDir = new File(base, "lib" + File.separator +
        folderName);
model = new Model(modelDir.getAbsolutePath());

```

8.2 Hallucinations du modèle Vosk

Bug — Mots hors vocabulaire reconnus par Vosk

Symptôme : le modèle retournait des mots absents du vocabulaire attendu, déclenchant des actions involontaires.

Cause : la grammaire JSON Vosk seule ne bloque pas toutes les approximations phonétiques hors dictionnaire.

Fix — Double filtre Grammaire Vosk + Liste blanche Java

```

if (!text.isEmpty() && ALLOWED_COMMANDS.contains(text))
    listener.onResult(text);
else if (!text.equals("[unk]") && !text.isEmpty())
    System.out.println("[VOSK] Mot hors dictionnaire rejete : "
        + text);

```

8.3 Superposition de flux audio

Bug — Lectures audio simultanées lors d'une navigation rapide

Symptôme : survoler rapidement plusieurs boutons lançait plusieurs threads audio en parallèle, produisant des sons superposés et des ressources non libérées.

Cause : chaque appel à playSound() lançait un nouveau ProcessBuilder sans vérifier l'existence d'un processus audio en cours.

Fix — Interruption du processus courant avant nouvelle lecture

```

if (currentAudio != null && currentAudio.isAlive())
    currentAudio.destroy();

// toggleTTS() interrompt également la lecture en cours :
public static void toggleTTS() {
    ttsEnabled = !ttsEnabled;
    if (!ttsEnabled && currentAudio != null)
        currentAudio.destroy();
}

```

8.4 Conflits Git entre Eclipse et IntelliJ IDEA

Bug — Conflits de structure de projet en gestion d'équipe

Symptôme : après un `git pull`, le projet ne s'ouvrait plus correctement dans l'IDE. Des répertoires parasites (`.idea/`, `*.iml`, `.classpath`, `.project`, `.settings/`) étaient pushés par certains membres, et la structure de sources (`src/`) différait selon que le projet avait été créé sous Eclipse ou sous IntelliJ. Lors des `fetch/pull`, Git créait parfois la structure d'un autre membre *à côté* de la structure existante, rendant le projet non importable dans l'IDE local.

Cause : absence d'un `.gitignore` commun dès le début du projet, et méthode de création de projet différente selon l'IDE (Maven-style sous IntelliJ vs. Java Project sous Eclipse).

Fix — .gitignore unifié et structure de sources harmonisée

Adoption d'un `.gitignore` partagé ignorant tous les fichiers spécifiques aux IDEs :

```
# Eclipse
.classpath
.project
.settings/
bin/

# IntelliJ IDEA
.idea/
*.iml
out/

# Commun
*.class
*.log
```

La structure de sources a été harmonisée autour d'un layout commun (`src/` à la racine, packages Java standards) reconnu par les deux IDEs sans configuration supplémentaire. Une convention de branche (`main` + branches fonctionnelles) a été adoptée en milieu de projet pour éviter les merges destructifs.

Conclusion

Ce projet intégré a permis de mettre en pratique l'ensemble du cycle de développement d'une application Java orientée objet : conception UML, implémentation des classes métier avec héritage et interfaces, persistance JDBC/MySQL, interfaces graphiques Swing et intégration de bibliothèques tierces (Vosk, TTS).

L'accent mis sur l'accessibilité — thème WCAG AAA, synthèse vocale contextualisée par sous-dossiers, reconnaissance vocale hors-ligne — confère à cette plateforme une dimension inclusive forte, directement alignée avec les enjeux réels de l'UIR pour ses étudiants en situation de handicap. Des fonctionnalités comme la navigation vocale ou le thème haute visibilité ont été pensées de manière autonome, au-delà du cahier des charges initial ; elles illustrent la capacité du groupe à s'approprier une problématique réelle et à proposer des solutions innovantes.

Les principaux apports techniques de ce projet sont :

- La maîtrise du pattern héritage/polymorphisme avec les hiérarchies `Utilisateur` → `{PSH, Administrateur}` et `Dossier` → `{Demande, Reclamation}` ;
- L'utilisation d'interfaces Java (`IArchivable`, `IStatistique`) pour définir des contrats comportementaux indépendants de la hiérarchie ;
- La séparation claire entre modèle, contrôleurs et vues ;
- L'intégration d'une reconnaissance vocale locale avec double filtre et une gestion audio multiplateforme à architecture sous-dossiers ;
- La gestion collaborative d'un dépôt Git multi-IDEs (Eclipse et IntelliJ IDEA) dans un contexte de travail en équipe.

Sur le plan organisationnel, ce projet aura été une expérience formatrice dans la gestion de contraintes réelles : coordination d'une équipe de huit personnes, IDEs hétérogènes, délais serrés en parallèle d'autres charges académiques. La réussite de ce déploiement nous a confortés dans l'idée qu'une plus grande autonomie—notamment une période de travail dédiée post-partiels, avec des libertés accrues dans les choix d'architecture et d'outils—aurait permis d'aller encore plus loin dans l'innovation et l'auto-apprentissage.

Des améliorations futures pourraient inclure : le hachage des mots de passe (bibliothèque `bcrypt` déjà présente dans le projet), un pattern DAO formel pour chaque entité, des notifications email lors des changements de statut, et une interface web en complément du client lourd Swing.

Références

- **Encadrement** : Mme. Guermah, Mme. Hanifi, M. Rahhal, *Projet Intégré — Plateforme PSH*, 3A, UIR ESIN, 2025-2026.
- **Cours** : Prof. Mehdi NAJIB, *Programmation Orientée Objet*, 3A - S5, UIR, 2025-2026.
- **Cours** : Prof. Olivier GRUBER, Responsable de l'UE POO, *Programmation Orientée Objet*, 3A - S5/S6, Polytech Grenoble INP 2024-2025
- **Cours** : Prof. Michael PERIN & Prof. Olivier GRUBER, Responsables de l'UE GPI, *Gestion de Projet Informatique*, 3A - S5/S6, Polytech Grenoble INP 2024-2025
- **Java SE** : Oracle, *Java SE 11 API Documentation*, <https://docs.oracle.com/en/java/javase/11/>
- **Vosk** : Alpha Cephei, *Vosk Speech Recognition Toolkit*, <https://alphacephei.com/vosk>
- **MySQL** : Oracle, *MySQL 8.0 Reference Manual*, <https://dev.mysql.com/doc/refman/8.0/>
- **WCAG** : W3C, *Web Content Accessibility Guidelines 2.1*, <https://www.w3.org/TR/WCAG21/>
- **Assistant IA** : Claude (Sonnet 4.6), Anthropic, <https://claude.ai>

Projet Intégré — Plateforme PSH — UIR
Année universitaire 2025-2026